MONITOR Z · L01.1

Clase 1 — La forma de un programa: renglones, columnas y sentencias

Cuaderno de estudio · versión del 2026-10-28

Margen derecho y renglones libres para tus notas. Las láminas son las mismas de la carpeta **_visual**. Si una lección se corrige, se regenera este cuaderno con la fecha nueva.

L01.1 — Clase 1 · La forma de un programa: renglones, columnas y sentencias

Temario de la L01: 1. La forma de un programa · 2. Los registros y las instrucciones de todos los días · 3. Entrar y salir: prólogo, save area, epílogo y RC · 4. Los datos: DS, DC, literales, el PARM y el `EX` · 5. Hablar con el sistema y leer el listado: macros, SVC, DCB, AMODE/RMODE.

Objetivo de esta clase: mirar un programa Assembler y saber **qué es cada renglón** antes de entender qué hace. Al terminar tenés que poder:

- decir en qué columna va cada cosa;
- distinguir una instrucción de una directiva y de una macro;
- partir un programa en sus cinco partes.

Usamos **tu** programa: `ZTKHOLA`, el de la práctica P01, que escribiste y corriste el 07-10 (JOB24962).

> 📚 Fuentes: - `monitor_z/asm/ZTKHOLA.asm`: 97 renglones. Los números de renglón de esta clase son de ese archivo. - El listado del ensamblado de **JOB24962** (`monitor_z/salidas/JOB24962/ASM/SYSPRINT`), del 2026-10-07 10:41: 170 sentencias y 681 bytes, sin ningún error. Los números de **sentencia** (`Stmt`) son de ese listado. - El JESMSGLG y el mapa del binder (`LKED/SYSPRINT`) del mismo job. - Para comparar: el listado de **JOB34775** (ZTKMON v0.9, 2026-10-22). - *HLASM Language Reference* (SC26-4940), capítulo 2 «Coding and structure»: los campos y las columnas.

1. Un programa es una pila de renglones de 80 columnas

Analogía: un **formulario de papel con casilleros**. Cada renglón tiene casilleros fijos: en el primero va el nombre, en el segundo qué hacer y en el tercero con qué. Si escribís fuera del casillero, el que lo lee se confunde.

El fuente vive en `PSJB.ZTK.ASM(ZTKHOLA)`, un PDS de registros fijos de **80 bytes**. Es la herencia de la tarjeta perforada: cada renglón es una tarjeta. El ensamblador lee los renglones de a uno, de arriba abajo.

Columnas	Campo	Para qué	En ZTKHOLA
1	**nombre** (etiqueta)	le pone nombre a ese lugar del programa, para poder saltar ahí o usar el dato	`ABRIO`, `SALIR`, `RCFINAL`
después de un blanco	**operación**	qué hacer	`STM`, `DS`, `OPEN`
después de un blanco	**operandos**	con qué hacerlo, separados por comas y **sin blancos**	`14,12,12(13)`
después de un blanco	**comentario**	texto libre para el que lee	`GUARDO REGISTROS DEL QUE LLAMA`
72	**continuación**	cualquier carácter que no sea blanco = «este renglón sigue en el próximo»	la `X` del renglón 94
73–80	identificación	lo ignora el ensamblador (antes, el número de tarjeta)	vacías

Dos reglas de verdad y una costumbre:

- **Regla:** si la columna 1 tiene un blanco, el renglón **no tiene nombre**. Si tiene un `*`, el renglón **entero es comentario**.
- **Regla:** el texto llega como máximo hasta la columna **71**. Lo que pongas en la 72 no es texto: es la marca de continuación.
- **Costumbre:** operación en la columna **10** y operandos en la **16**. El ensamblador sólo exige un blanco entre campos, pero tus programas (ZTKHOLA, ZTKMON) y las macros de IBM que se expanden en el listado se alinean así, y se lee mucho mejor.

Tu renglón 12, medido columna por columna:

```
columna:  1        10    16                36
          |        |     |                 |
         [         STM   14,12,12(13)       GUARDO REGISTROS DEL QUE LL
          sin nombre  op   operandos           comentario
```

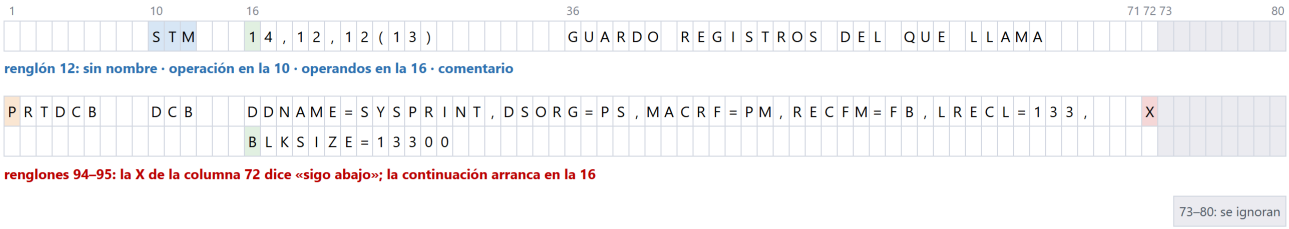
Un renglón que sigue en el próximo (renglones 94 y 95):

```
PRTDCB   DCB   DDNAME=SYSPRINT,DSORG=PS,MACRF=PM,RECFM=FB,LRECL=133,   X
               BLKSIZE=13300
```

> ⚠️ **Lo que aprendiste en la P01:** un comentario largo que llega a la columna 72 se convierte **sin querer** en una continuación, y el ensamblador pega el renglón siguiente al anterior. Está anotado en la P01 como lo aprendido del intento 1.

Un programa es una pila de renglones de 80 columnas

L01 · Clase 1 · La forma de un programa



Columnas	Campo	Para qué	En ZTKHOLA
1	nombre	le pone nombre a ese lugar, para saltar ahí o usar el dato	`ABRIO` · `RCFINAL`
tras un blanco	operación · operandos · comentario	qué hacer, con qué (sin blancos) y por qué	`STM 14,12,12(13)`
72	continuación	un carácter no blanco = «el renglón sigue»	la `X` del renglón 94

> ⚠️ **Lo que aprendiste en la P01:** un comentario que llega a la columna 72 se vuelve, sin querer, una continuación. `*` en la columna 1 = renglón entero de comentario.

Fuente: monitor_z/asm/ZTKHOLA.asm (renglones 12, 94, 95) · HLASM Language Reference, cap. 2 · P01 (intento 1)

Lámina L01-10_renglones_de_80

2. Tres clases de sentencias

Un renglón que no es comentario es una **sentencia**. Hay tres clases, y la diferencia está en **a quién le habla**:

Clase	A quién le habla	Qué produce	Ejemplo de ZTKHOLA	Qué salió en el listado
🔧 **Instrucción de máquina**	a la **CPU**	bytes de código: 2, 4 o 6	`STM 14,12,12(13)` (renglón 12)	`90EC D00C`: 4 bytes
📋 **Directiva** (*instrucción del ensamblador*)	al **ensamblador**	a veces nada; a veces reserva lugar	`USING ZTKHOLA,12` (renglón 14)	**ningún byte**: es una promesa al ensamblador

📦 **Macro**	a una **biblioteca de IBM**	varias sentencias nuevas, que a su vez son instrucciones y directivas	`OPEN (PRTDCB,OUTPUT)` (renglón 27)	**5 sentencias** con `+`, la última `SVC 19`

Las directivas que tiene ZTKHOLA:

Directiva	Qué le dice al ensamblador	Renglón
`CSECT`	«acá empieza un programa (una sección de control) que se llama ZTKHOLA»	1
`AMODE` / `RMODE`	con cuántos bits direcciona y dónde se puede cargar (clase 5)	2–3
`USING`	«en ejecución, R12 va a valer la dirección de ZTKHOLA» (clase 3)	14
`DS`	**reservá** lugar, sin valor inicial	33, 58, 64, 72, 83–86
`DC`	**reservá** lugar **con** un valor inicial	87
`LTORG`	«poné acá los literales `=F'8'`, `=C'…'`» (clase 4)	96
`END`	«se terminó el fuente, y el programa arranca en ZTKHOLA»	97

Las cinco macros: `OPEN`, `WTO`, `PUT`, `CLOSE` y `DCB`. Las trae `SYS1.MACLIB`: el listado lo dice en la página 8 (*Macro and Copy Code Source Summary*), que también lista `IHBINNRA`, `IHB01` y `SYSSTATE`, macros internas que usan esas cinco.

Tus 97 renglones, contados:

Clase	Renglones
💬 comentarios (`*` en la columna 1)	45
🔧 instrucciones de máquina	31
📋 directivas	15
📦 macros	5
🔁 continuación	1

total	**97**

Casi la mitad son comentarios. No sobran: en Assembler, el comentario es el **único** lugar donde queda escrito el **por qué**.

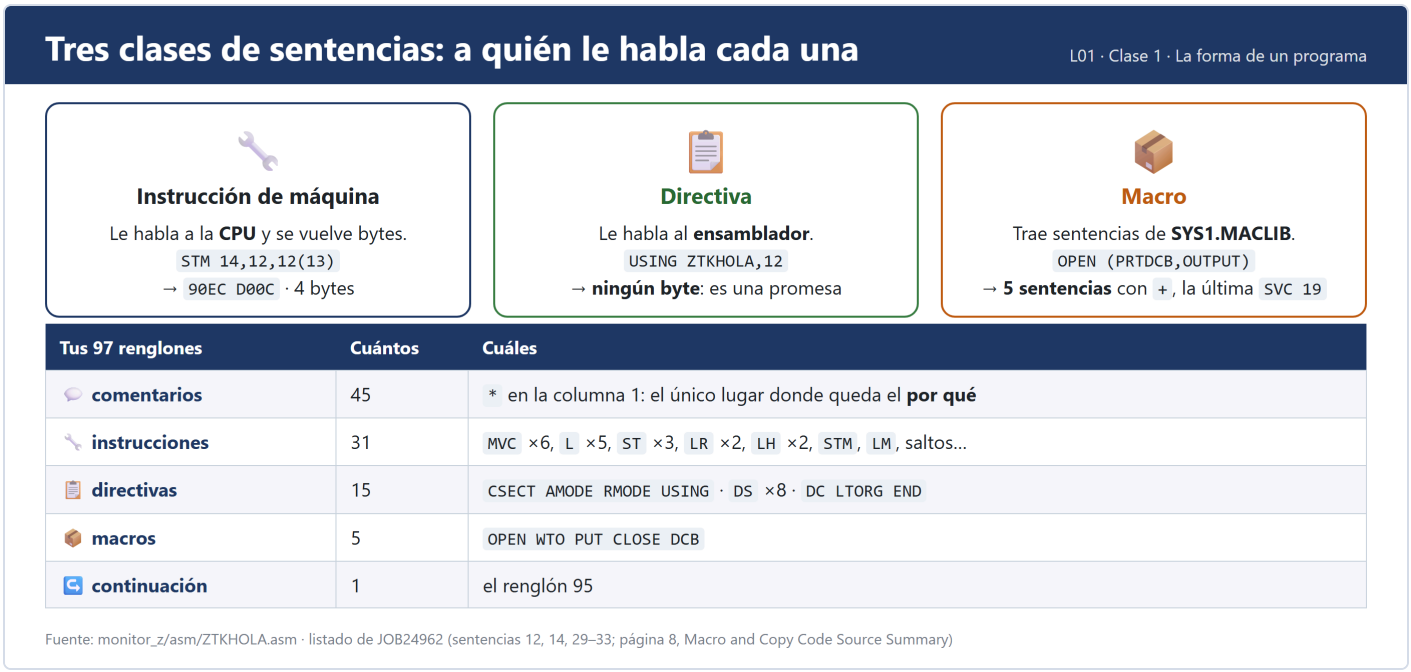


Lámina L01-11_tres_clases

3. Las cinco partes de un programa

Todos los programas que vas a ver, desde ZTKHOLA hasta ZTKMON, tienen las mismas cinco partes y en el mismo orden:

#	Parte	Qué hace	Renglones	Dónde quedó (columna Loc)	Bytes
1	🏷️ **Cabecera**	dice cómo se llama y cómo se carga, y un comentario que explica el programa	1–11	`000000` (no genera bytes)	0
2	🚪 **Prólogo**	guarda los registros del que llamó y prepara la base	12–20	`000000` – `00001D`	30
3	⚙️ **Cuerpo**	el trabajo: los 4 bloques (abrir, leer el PARM, escribir, cerrar)	21–71	`00001E` – `0000B9`	156

4	🚶 **Epílogo**	deja el RC en R15, restaura los registros y vuelve	72–78	`0000BA` – `0000CB`	18
5	🗄️ **Datos**	las áreas, el DCB, la instrucción modelo del `EX` y los literales	79–97	`0000CC` – `0002A8`	477
			97		**681**

Lo que dice la tabla:

- El código que se ejecuta (partes 2 a 4) ocupa **204 bytes**. Los datos ocupan más del doble (477).
- Los datos van **después** del `BR 14` del renglón 78. Ahí la CPU nunca llega, porque `BR 14` ya volvió al sistema. Si los datos estuvieran en medio del código, la CPU intentaría ejecutarlos como instrucciones.
- Lo más grande de los datos son dos renglones de 133 bytes (`SALIDA` y `ESPACIOS`), la save area (72) y el DCB (96).

> Las clases 3 y 4 entran en el prólogo, el epílogo y los datos.

Las cinco partes, siempre en el mismo orden

L01 · Clase 1 · La forma de un programa

Los 681 bytes de ZTKHOLA, por parte (columna Loc del listado)



Prólogo 30 · Epílogo 18 · código que se ejecuta: 204 bytes · datos: 477

	Parte	Qué hace	Renglones	Loc
🏷️	**Cabecera**	cómo se llama, cómo se carga, y qué hace el programa (comentario)	1–11	`000000 (0 bytes)`
🚪	**Prólogo**	guarda los registros del que llamó y prepara la base	12–20	`000000-00001D`
⚙️	**Cuerpo**	el trabajo: abrir, leer el PARM, escribir, cerrar	21–71	`00001E-0000B9`
🚶	**Epílogo**	RC en R15, restaurar y volver con BR 14	72–78	`0000BA-0000CB`
🗄️	**Datos**	áreas, DCB, la instrucción modelo del EX, literales	79–97	`0000CC-0002A8`

Los datos van **después** del `BR 14` (renglón 78): la CPU nunca llega ahí. En medio del código, intentaría ejecutarlos como instrucciones.

Fuente: listado de JOB24962 (columna Loc) · ZTKHOLA.asm

Lámina L01-12_cinco_partes

4. Del renglón a los bytes: cómo se lee el listado

El ensamblador no sólo genera el objeto: también imprime el **listado**, que pone **al lado de cada renglón en qué se convirtió**. Así se ve tu renglón 12 en JOB24962:

```
  Loc  Object Code    Addr1 Addr2  Stmt   Source Statement
000000 90EC D00C            0000C    12          STM   14,12,12(13)
```

Columna	Qué es	En el renglón 12
`Loc`	**dónde** queda, contado en hexadecimal desde el principio del programa	`000000` : es lo primero
`Object Code`	los **bytes** que generó	`90EC D00C` (4 bytes)
`Addr1` / `Addr2`	la dirección que calculó para cada operando	`0000C` : el desplazamiento 12, en hexadecimal
`Stmt`	el número de **sentencia**	12
`Source Statement`	tu renglón, tal cual	

Por qué el listado tiene 170 sentencias y no 97: cada macro se **expande**. El ensamblador va a buscarla a `SYS1.MACLIB` y pone en su lugar las sentencias que contiene, marcadas con un `+`. Tu `OPEN` del renglón 27 se convirtió en las sentencias 29 a 33:

```
000020 4D10 C028      00028    30+         BAS   1,*+8
000024 8F                      31+         DC    AL1(143)
000025 00022C                  32+         DC    AL3(PRTDCB)
000028 0A13                    33+         SVC   19
```

Con las 5 macros pasa lo mismo, y la que más crece es `DCB` (renglones 94–95), que se vuelve las sentencias 125 a 163: el bloque de 96 bytes que describe el archivo.

Al final del listado, el resumen (página 11):

Qué dice	Qué significa
`No Statements Flagged in this Assembly`	ningún error ni advertencia

`97 Primary Input Records Read`	leyó tus 97 renglones
`6413 Library Records Read`	y 6.413 renglones de las macros de IBM
`Processor Time: 00.00.00.0192`	19 milésimas de segundo de CPU
`Return Code 000`	el `ASM 00` que ves en el JESMSGLG

Después, el binder lo guardó en `PSJB.ZTK.LOAD(ZTKHOLA)` con `LENGTH = 2A9` (681 bytes), y anotó `MODULE SIZE (HEX) 000002B0`: 688 bytes. La diferencia de 7 bytes es relleno: no está medido por qué ese número exacto, pero 688 es el múltiplo de 8 más cercano por arriba. Cómo pasa del objeto al load module es la L15.1.



Lámina L01-13_leer_el_listado

5. Lo mismo, en grande: ZTKMON v0.9

	ZTKHOLA (JOB24962)	ZTKMON v0.9 (JOB34775)
Renglones del fuente	97	2.555
Sentencias en el listado	170	13.024

Bytes del programa	681 (`X'2A9'`)	63.276 (`X'F72C'`)
CPU del ensamblado	0,0192 s	0,1227 s
Errores	ninguno	1 sentencia marcada, con severidad 0 (un aviso, no un error)
Return code	000	000
Las cinco partes	✅	✅ las mismas, con un cuerpo de muchas rutinas

ZTKMON es **26 veces más largo** en renglones, pero tiene la misma forma. Si sabés leer ZTKHOLA, sabés **por dónde** leer ZTKMON: la cabecera arriba, el prólogo después, las rutinas en el medio y los datos al final.

Resumen

Idea	En una línea	Testigo
Renglones de 80	nombre en la columna 1, la continuación en la 72, y 73–80 se ignoran	ZTKHOLA, renglones 12 y 94
Comentario	`*` en la columna 1 = el renglón entero es comentario: 45 de 97	ZTKHOLA
Tres clases	instrucción → bytes; directiva → le habla al ensamblador; macro → se expande	listado de JOB24962
Cinco partes	cabecera, prólogo, cuerpo, epílogo y datos, siempre en ese orden	204 bytes de código, 477 de datos
El listado	muestra al lado de cada renglón su `Loc` y sus bytes; los `+` son macros expandidas	97 renglones → 170 sentencias
El resumen	`No Statements Flagged` y `Return Code 000`	JOB24962, página 11

Preguntas de comprobación

1. En el renglón 94 la `X` está en la columna 72. Si la corrieras a la columna 71, ¿qué pasaría con el renglón 95? ¿Qué creés que diría el listado?
2. `USING` no genera ningún byte y `DS` tampoco escribe nada, pero `DS` hace que el `Loc` avance y `USING` no. ¿Por qué uno ocupa lugar y el otro no?

Mirá `SAVEAREA DS 18F`: ¿cuántos bytes avanzó el `Loc` (de `0000D4` a `00011C`)?

3. Elegí un renglón del bloque 2 (renglones 40 a 44) y escribí sus columnas `Loc` y `Object Code`, sacándolas del listado de JOB24962. ¿Cuántos bytes ocupa cada instrucción?

Mis respuestas

Relacionado: *L01 - Anatomia de un programa Assembler - el prologo de ZTKMONS* · *P01 - ZTKHOLA* · *L15.1 - Clase 1 - Del fuente al programa que corre* · *Ruta de aprendizaje*.

Notas